Rishabh Indoria
SR No. 24002
M.Tech AI

Digital Image Processing

November 10, 2024

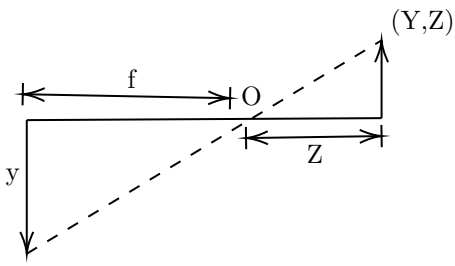
1 Evaluative components

- 5 **programming** assignments, at least 2 to be done in python. **30%**
- **Project**: Study a new topic or explore deeper on a topic taught in class(Presentation, demo, & viva). **20%**
- 2 Midterms and 1 Final exam for **50%**

2 Basics of Image formation and Quantization

2.1 Image Formation

- Image is projection of a 3D object on a 2D surface, thus 3D information and getting it back is very, very hard.
- **Pinhole Projection Model**: Lens is modelled as a pinhole through which all light rays will pass. The image plane is located one focal length (f) away from the lens. Image is recorded using sensors on the image plane.



$$\frac{Z}{f} = \frac{Y}{y} \implies y = \frac{fY}{Z}$$

Figure 1: Pinhole Projection Model

- Difference between true distance of points and the distance on the image plane is very different.
- For pinhole model, straight lines in 3D map to straight lines in 2D. This is not true for other projection models, for example fish eye lens.

2.2 Sampling and Quantization

- **Sampling**: The continuous signal is sampled at a discrete number of points in the plane.
- **Quantization**: Each sample is represented using a finite number of bits.(Generally 8 bits)
- Sampled image is a matrix of numbers representing image intensities. Each element is called **a picture element** or **pixel**. Images can be square or non-square.
- **Intensity** is represented as a gray level, i.e. $\{0, 1, 2, ..., K-1\}$, where $K = 2^b$, for a grayscale image. For RGB images, we have 3 matrices, each representing intensities of one color.

2.3 Image Notation

- Image is denoted as a $M \times N$ matrix $I(i, j)$, where $i \in \{0, ..., M-1\}$ and $j \in \{0, ..., N-1\}$.
- For a gray scale image $I(i, j) \in \{0, 1, ..., 255\}$, I represents pixel intensities, $K = 256$(total no. of intensities).
- Alternatively, we could represent images using normalized intensities, $I(i, j) \in \{0, \frac{1}{256}, \frac{2}{256}, ..., 1\}$.

2.4 Image Formats

- **.bmp**: Bitmap format, no compression or no loss of information
- **.png**: Portable Network Graphic, lossless compression
- **.tiff**: Tagged Image File Format, lossless compression
- **.jpg**: Joint Photographic Exports Gap, lossy compression, same as **.jpeg**

3 Binary Images

If $I(i,j) \in \{0,1\}$ or $\{0,255\}$, then the image is a **binary image**.
Binary images are obtained by thresholding gray scale images.
Simple Thresholding: If the threshold is T, for a given image I the binary image J is represented as

$$J(i,j) = \begin{cases} 0, & I(i,j) \leq T \\ 1, & I(i,j) > T \end{cases} \tag{1}$$

Gray level Image Histogram: Histogram H of image I is defined as $H : \{0,1,...,K-1\} \to \{0,1,...,M \times N\}$
$\mathbf{H(k)}$ = Number of occurrences of gray level k in the image, $k \in \{0,1,...,255\}$

3.1 Otsu's Binarization

- An algorithm used to determine T from an image I automatically.
- **Normalized Histogram**: $p(k) = \frac{H(k)}{MN}$, can be regarded as a probability mass function.
  We interpret $p(k)$ as the probability that a pixel takes value k.
- **Class Probabilities**
  $p(C_0) = \omega_0 =$ Probability that pixel belongs to $C_0 = \sum_{k=0}^{t} p(k)$
  $p(C_1) = \omega_0 =$ Probability that pixel belongs to $C_1 = \sum_{k=t+1}^{K-1} p(k)$
- **Class Means**
  $\mu_0 =$ Mean values of a pixel given that pixel belongs to class $C_0 = \sum_{k=0}^{t} kp(k|C_0)$
  $p(k|C_0) =$ Probability that pixel takes value k given that pixel belongs to $C_0 = \frac{p(k,C_0)}{p(C_0)} = \frac{p(k)p(C_0|k)}{p(C_0)}$
  $p(C_0|k)$ is 1 when $k \in \{0,...,t\}$ else 0
  $\implies \mu_0 = \frac{1}{p(C_0)} \sum_{k=0}^{t} kp(k) = \frac{1}{\omega_0} \sum_{k=0}^{t} kp(k)$
  Similarly, we define $\mu_1 = \frac{1}{\omega_1} \sum_{k=t+1}^{K-1} kp(k)$
- **Overall Mean**: $\mu_T = \sum_{k=0}^{K-1} kp(k) = \sum_{k=0}^{t} kp(k) + \sum_{k=t+1}^{K-1} kp(k) = \mu_0\omega_0 + \mu_1\omega_1$
- **Class Variances**
  $\sigma_0^2 = \sum_{k=0}^{t} (k-\mu_0)^2 p(k|C_0) = \sum_{k=0}^{t} \frac{(k-\mu_0)^2 p(k)}{\omega_0}$
  $\sigma_1^2 = \sum_{k=t+1}^{K-1} (k-\mu_1)^2 p(k|C_1) = \sum_{k=t+1}^{K-1} \frac{(k-\mu_1)^2 p(k)}{\omega_1}$
- **Image Variance**: $\sigma_T^2 = \sum_{k=0}^{K-1} (k-\mu_T)^2 p(k)$
- **Intra Class Variance**: $\sigma_w^2 = \omega_0\sigma_0^2 + \omega_1\sigma_1^2$
- **Goal of thresholding is to minimize** σ_w^2. Known as brute force search. Since, we are calculating second order statistics it becomes very complex and time-consuming.
- **Inter Class Variance**: $\sigma_B^2 = \omega_0(\mu_0 - \mu_T)^2 + \omega_1(\mu_1 - \mu_T)^2$
- It can be shown that $\sigma_B^2 + \sigma_w^2 = \sigma_T^2$, as such, instead of minimizing, σ_w^2 we can **maximize** σ_B^2.
- Simplifying $\sigma_B^2 = \omega_0\omega_1(\mu_0 - \mu_1)^1$

3.2 Connected Component Analysis

- Let's consider 4 neighbors of (x, y), they can be written as $N_4(x, y) = \{(x+1, y), (x-1, y), (x, y-1), (x, y+1)\}$
- (x_1, y_1) and (x_2, y_2) are 4-**adjacent** if $(x_1, y_1) \in N_4(x_2, y_2)$ or $(x_2, y_2) \in N_4(x_1, y_1)$.
- Path from (x, y) to (s, t) is a sequence of pixels, where each successive pair is 4-adjacent.
- (x, y) and (s, t) are connected if there exists a path of pixels from (x, y) to (s, t) such that all these pixels are of the same polarity.
- A **connected component** is a set of pixels such that any pixel in this set is connected to any other pixel in the same set.

Algorithm 1 Extraction of Connected Components

```
R = 0 (Region index array)
k ← 1 (Region index counter)
for i = 0 to n do
    for j = 0 to m do
        if I(i, j) == 0 then
            do nothing
        end if
        if I(i − 1, j) == 0 & I(i, j − 1) == 0 then
            R(i, j) ← k
            k ← k + 1
        end if
        if I(i − 1, j) == 1 & I(i, j − 1) == 0 then
            R(i, j) ← R(i − 1, j)
        end if
        if I(i − 1, j) == 0 & I(i, j − 1) == 1 then
            R(i, j) ← R(i, j − 1)
        end if
        if I(i − 1, j) == 1 & I(i, j − 1) == 1 then
            R(i, j) ← R(i − 1, j)
            if R(i − 1, j) ≠ R(i, j − 1) then
                Make the regions same
            end if
        end if
    end for
end for
```

3.3 Binary Filters

- **Binary morphology** - Change in shapes
- **Window**: Set of neighbors of a pixel according to a geometric rule. 1D window would include: Row(3), Column(3), Row(5).
  2D windows would be Square(9), Cross(5)
- **Window Notation**: Window $B = \{B_1, B_2, ..., B_{2p+1}\}$ where $B_i = (p_i, q_i)$ such that B contains center pixel $(0, 0)$.
- **Window Set**: Given window B and image I, $B \Diamond I(i, j) = \{I(i + p, j + q, (p, q) \in B)\}$.
- **Binary Filter**: Let G be a binary operation on the windowed set. The filtered image is obtained as $J(i, j) = G[B \Diamond I(i, j)]$.
- **Edge of image processing**: One solution is to repeat as many boundary pixels as required for the window.
- **Dilation Filter**: $J = DILATE(I, B)$ is defined as $J(i, j) = OR\{B \Diamond I(i, j)\}$, where $OR\{x_1, ..., x_n\} = x_1 + ... + x_n = max(x_1, ..., x_n)$
- **Erosion Filter**: $J = ERODE(I, B)$ is defined as $J(i, j) = AND\{B \Diamond I(i, j)\}$, where $AND\{x_1, ..., x_n\} = x_1.x_2...x_n = min(x_1, ..., x_n)$
- Dilation and Erosion are dual operators, $\overline{x_1 + x_2} = \overline{x_1}.\overline{x_2}$.
- **Majority Filter**: $J = MAJORITY(I, B)$, helps remove random noise.
- **Boundary Detection**: $XOR\{I, ERODE(I, B)\}$

4 Gray Scale Image Processing

4.1 Point Operations

- **Point Operations**: $J(i,j) = f(I(i,j))$ applied $\forall i,j$. This does not depend on local neighborhood.
- **Image Offset**: $J(i,j) = I(i,j) + L$, where L is a constant identity.
  if $L > 0$, then J is a brightened version of I and
  if $L < 0$, then J is a darkened version of I
  Application: $I_1, I_2, ..., I_n$ are images of some scene captured using different exposures. We wish to equalize all their average intensities to $\frac{K}{2}$, where K is the number of intensity levels.
  Let each image I_m have mean μ_m, then $J_m(i,j) = I_m(i,j) - \mu_m + \frac{K}{2}$
- **Image Scaling**: $J(i,j) = PI(i,j)$, where $P > 0$.
  if $P > 1$, then the intensity range is widened and
  if $P < 1$, then the intensity range is shrunk.
- **Offset & Scaling**: $J(i,j) = PI(i,j) + L$
  For **negative image**, $P = -1$ and $L = K - 1$, $J(i,j) = K - 1 - I(i,j)$
  Full scale contrast stretch, let A and B denote min & max gray levels. We choose P and L such that $PA + L = 0$ and $PB + L = K - 1$
  We get $P = \frac{K-1}{B-A}$ and $L = -\frac{(K-1)}{B-A}A$, $J(i,j) = \frac{K-1}{B-A}(I(i,j) - A)$.
- **Non-Linear Point Operations**: $J(i,j) = f(I(i,j))$
  $f(x)$ can be $x^2, \sqrt{x}, ce^x, \log 1 + x$ or x^r where $r > 0$. The last one is called gamma correction.

4.2 Histogram Equalization

- Flat histograms make rich use of the entire gray scale.
- Goal is to process the image to obtain an approximately flat histogram.
- **Continuous Case**
  Let $p(x) \to pdf \equiv$ normalized histogram
  and $P(x) \to cdf$
  Let *pdf* and *cdf* of J be $q(x)$ and $Q(x)$. We want $q(x) = 1$ and $Q(x) = x$.
  $J(i,j) = P(I(i,j))$ is the transformation, $Q(x) = Pr\{J \le x\} = Pr\{P(I) \le x\} = Pr\{I \le y\} = P(y)$.
  In a flat region, $Pr(I \le y) = x, \forall y : P(y) = x$.
  Example: Let $p(x) = 2x \implies P(x) = x^2$, and $J(I) = P(I) = I^2$.
  $Q(x) = Pr[J \le x] = Pr[I^2 \le x] = Pr[I \le \sqrt{x}] = P(\sqrt{x}) = x$
- **Discrete Case**
  $J = P(I) = Pr(P \le I) = \sum p(u)$ where $p(u)$ is normalized histogram.
  $J(i,j) = \sum_{u=0}^{I(i,j)} p(u)$.

  Example: Let the image be

1	1	3	4
2	5	3	2
8	1	8	2
4	5	3	11

k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$h(k)$	0	3	3	3	2	2	0	0	2	0	0	1	0	0	0	0
$p(k)$	0	$\frac{3}{16}$	$\frac{3}{16}$	$\frac{3}{16}$	$\frac{2}{16}$	$\frac{2}{16}$	0	0	$\frac{2}{16}$	0	0	$\frac{1}{16}$	0	0	0	0
$P(k)$	0	$\frac{3}{16}$	$\frac{6}{16}$	$\frac{9}{16}$	$\frac{11}{16}$	$\frac{13}{16}$	$\frac{13}{16}$	$\frac{13}{16}$	$\frac{15}{16}$	$\frac{15}{16}$	$\frac{15}{16}$	1	1	1	1	1

  $J = P(I)$ to discretize the values, we multiply by $K - 1 = 15$

$15 \times P(k)$	0	3	6	8	10	12	12	12	14	14	14	15	15	15	15	15

  Resultant image is

3	3	8	10
6	12	8	6
14	3	14	6
10	12	8	15

  If we don't want to do all the calculations, a rough estimate can be obtained by directly calculating *cdf* without normalizing then do $\frac{CDF(x) - CDF_{min}}{N - CDF_{min}} \times (L - 1)$, where CDF_{min} is the minimum non-zero *cdf*.

4.3 Adaptive Histogram Equalization

- We apply Histogram equalization on patches of the original image.
- Patches based methods leads to boundary artifact or blocking artifact.
- **Boundary artifact**: Similar intensity pixels may get mapped to entirely different intensities.
- To solve this, we can use overlapping patches and then take the average on overlapped pixels.

4.4 Contrast Limited Adaptive Histogram Equalization

- In smooth regions with noise, adaptive histogram equalization tends to amplify the noise.
- This is solved by **CLAHE**.
- We use a hyperparameter, **Clip limit**, that clips the histogram and moves the mass above or beyond the clip to other intensities. Essentially flattening the histogram slightly before equalization.
- We end up with a flatter input histogram that limits contrast enhancement through equalization.

4.5 Histogram Matching/Specification

- If I has CDF $P(x)$, then $J = P(I)$ gives a histogram equalized image.
- Suppose we have an image J with uniform distribution, then $P^{-1}(J)$ will give an image with CDF $P(x)$.
  CLAIM: $P^{-1}(J)$ will have CDF $P(x)$ if J has uniform distribution
  $Pr\{P^{-1}(J) \leq x\} = Pr\{J \leq P(x)\} = P(x)$
- Given an image I with CDF $P(x)$, we want to transform I to J such that J has CDF $Q(x)$. This can be achieved by $J = Q^{-1}(P(I))$.
- However, this is not viable for digital images. Instead, we try to do $Q(J) = P(I)$
  or $k^*(i,j) = \text{argmin}_k |Q(k) - P(I(i,j))|$

Algorithm 2 Histogram Matching

```
J ← Empty matrix of same size as I
sum ← 0 (This would be P(I))
▷ Assume I[i][j] is normalized
for i = 0 to n do
    for j = 0 to m do
        Q_min ← ∞ (Current Minimum Q)
        J[i][j] ← −1 (Current argmin k*)
        sum ← sum + I[i][j]
        for k = 0 to K − 1 do
            Q = Σ_{l=0}^{k} q(l)
            if Q − sum < Q_min then
                Q_min ← Q − sum
                J[i][j] ← k
            end if
        end for
    end for
end for
```

4.6 Advanced Contrasting techniques

- These can be used as project topics
- **Retinex Theory**: An image as a product of illumination and reflectance from the object, $I = LR$. This works well for low light images.
- **Dark Channel Prior**: Hazy image's dark pixels, i.e., the pixels that receive immensely low-intensity values, provide a definite estimation of the transmission of the haze, $J = tI + (1-t)A$, I is haze free image, t represent transmission map, and A represents global atmospheric light.
- **Deep Learning Based Methods**

5 Spatial Operators

5.1 Background

- **Linear Systems and Filters**: Let's say T is a linear system, and image is represented as $m \times n$ matrix. Take two images $f_1(m,n)$ and $f_2(m,n)$. A linear system satisfies $T[f_1(m,n) + f_2(m,n)] = T[f_1(m,n)] + T[f_2(m,n)]$, and $T[\alpha f_1(m,n)] = \alpha T[f_1(m,n)]$
- **Time/Shift Invariant System**: Let $g(m,n) = T[f(m,n)]$, for some shift invariant system $T[f(m-k,n-l)] = g(m-k,n-l)$. Shifting the image shifts the output.
- **Discrete Impulse Function**

$$\delta(m,n) = \begin{cases} 1, & m = 0, n = 0 \\ 0, & \text{elsewhere} \end{cases} \tag{2}$$

- **Linear Time/Shift Invariant System**: $f(m,n) \to \boxed{\text{T}} \to g(m,n)$, for impulse function $\delta(m,n) \to \boxed{\text{T}} \to h(m,n)$ [Impulse Response]

$$f(m,n) = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f(k,l)\delta(m-k,n-l) \tag{3}$$

$$\begin{aligned} g(m,n) = T(f(m,n)) &= T(\sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f(k,l)\delta(m-k,n-l)) \\ &= \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f(k,l)T(\delta(m-k,n-l)) \\ &= \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f(k,l)h(m-k,n-l) = f * h \end{aligned} \tag{4}$$

  Convolving f with h corresponds to passing an image through an LSI system with impulse response h. If the system is LSI, then the output is related to the input by convolution, and vice versa.
- **Correlation Operator**: $g(m,n) = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f(m+k,n+l)h(k,l)$
- Convolution is commutative. If $h(m) = h(-m)$, then convolution and correlation are the same.
- In practice, correlation is used since it's convenient.
- Deal with boundaries by using 0 padding or replicating.
- For a signal of length M and filter of size $2K+1$ ($M >> K$), then $M - 2K$ samples have no boundary artifacts.

5.2 Smoothing Filters

- $h_1(m,n) = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$, and $h_2(m,n) = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$
- $h_3(m,n) = ke^{-(m^2+n^2)}$, where $m,n \in \{-1,0,1\}$. k is chosen such that $\sum h_3 = 1$.
- We choose k such that $f(m,n) * h(m,n) = f(m,n)$ in smooth regions.
- Effect of smoothing is blurring.
- Denoising or smoothing before binarization to reduce noise.
- Smoothing can also be achieved using non-linear filters.

5.3 Sharpening Filters

- For all purposes $f(x, y) \equiv f(m, n)$
- $\frac{\partial f}{\partial x} = f(m+1, n) - f(m, n)$, and $\frac{\partial f}{\partial y} = f(m, n+1) - f(m, n)$
- $\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x}(\frac{\partial f}{\partial x}) = f(m+1, n) - f(m, n) - (f(m, n) - f(m-1, n)) = f(m+1, n) + f(m-1, n) - 2f(m, n)$, and $\frac{\partial^2 f}{\partial y^2} = f(m, n+1) + f(m, n-1) - 2f(m, n)$
- Laplacian of an image $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$
- Another variant is $\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
- Highlights intensity discontinuations or de-emphasizes regions with slowly varying intensity levels.
- To enhance the image(edge enhancement or sharpening), $g(x, y) = f(x, y) + c\nabla^2 f(x, y)$
  $c = -1$, if central coefficient of $\nabla^2 f(x, y)$ is $-$ve
  $c = +1$, if central coefficient of $\nabla^2 f(x, y)$ is $+$ve

5.4 Unsharp masking and High Boost filtering

- $\bar{f}(x, y)$ is a blurred version of $f(x, y)$(unsharp version).
- Define $g_{mask}(x, y) = f(x, y) - \bar{f}(x, y)$, and $g(x, y) = f(x, y) + kg_{mask}(x, y)$
- If $k = 1$, then it is unsharp masking
- If $k > 1$, then it is high boost filtering

5.5 Geometric Operators

Input image is $I(i, j)$
Output image is $J(i, j) = I(a_1(i, j), a_2(i, j))$

1. Image Translation
   - $a_1(i, j) = i - b_1$, and $a_2(i, j) = j - b_2$
   - Results in a $M + |b_1| \times N + |b_2|$ sized image
2. Image Rotation
   - $a_1(i, j) = i \cos\theta - j \sin\theta$, and $a_2(i, j) = i \sin\theta + j \cos\theta$
   - Rotates by θ relative to horizontal axis in 'counter' clockwise direction.
   - i_{min}: smallest value of i(in image J) for which you are able to read off from I.
     j_{min}: smallest value of j(in image J) for which you are able to read off from I.
   - $J'(i, j) = J(i + i_{min}, j + j_{min})$, results in $[(i_{max} - i_{min}) \times (j_{max} - j_{min})]$
3. Image Zoom
   - $a_1(i, j) = \frac{i}{b_1}$, and $a_2(i, j) = \frac{j}{b_2}$
   - If $b_1 b_2 \geq 1$, then it is super resolution or magnification
   - If $b_1 b_2 < 1$, then it is downsampling or minification
   - Need to interpolate when a_1 and a_2 are not integers.
4. Nearest Neighbor Interpolation
   - a_1 and a_2 are mapped to the nearest integer coordinates
   - $J(i, j) = I(i', j')$, where $i' = round(a_1)$ and $j' = round(a_2)$
5. Bilinear Interpolation
   - Given 4 nearest neighbors of a_1, a_2: $I(i_1, j_1), I(i_2, j_2), I(i_3, j_3), I(i_4, j_4)$.

- Define $J(i,j) = A_0 + A_1 i + A_2 j + A_3 ij$
- $\begin{bmatrix} I(i_1,j_1) \\ I(i_2,j_2) \\ I(i_3,j_3) \\ I(i_4,j_4) \end{bmatrix} = \begin{bmatrix} 1 & i_1 & j_1 & i_1 j_1 \\ 1 & i_2 & j_2 & i_2 j_2 \\ 1 & i_3 & j_3 & i_3 j_3 \\ 1 & i_4 & j_4 & i_4 j_4 \end{bmatrix} \begin{bmatrix} A_0 \\ A_1 \\ A_2 \\ A_3 \end{bmatrix}$
- Suppose we have a smooth region, then $A_0 = c, A_1 = A_2 = A_3 = 0$
- In general $J(x,y) = B_{11} I(x_1,y_1) + B_{12} I(x_1,y_2) + B_{21} I(x_2,y_1) + B_{22} I(x_2,y_2)$, where $B_{11} = \frac{x_2 - x}{x_2 - x_1} \frac{y_2 - y}{y_2 - y_1}$.

6 Fourier Transforms

6.1 Introduction

- **Sinusoidal Signals** are of the form $f(x) = \cos \Omega x$ or $\sin \Omega x$ where $x, y \in \mathbb{R}$
- **Complex Exponential** is $f(x) = e^{j\Omega x} = \cos \Omega x + j \sin \Omega x$
- **Dirac Delta function**

$$\delta(x) = \begin{cases} \infty, & x = 0 \\ 0, & x \neq 0 \end{cases} \qquad \int_{-\infty}^{\infty} \delta(x) dx = 1 \; \delta(cx) = \frac{1}{|c|} \delta(x)$$

$$\int_{-\infty}^{\infty} \delta(x) f(x) dx = f(0) \text{ when } f(x) \text{ is continuous at } x = 0$$

$$\int_{-\infty}^{\infty} \delta(x - x_0) f(x) dx = f(x_0)$$

$$\frac{1}{2\pi} \int_{-\infty}^{\infty} e^{-j\Omega x} dx = \delta(\Omega)$$

- **Kronecker Delta function**

$$\delta(m) = \begin{cases} 1, & m = 0 \\ 0, & m \neq 0 \end{cases}$$

6.2 Continuous Fourier Transform

- Fourier transform of function $f(x)$ can be obtained by

$$F(\Omega) = \int_{-\infty}^{\infty} f(x) e^{-j\Omega x} dx$$

- Sufficient conditions for the existence of fourier transform is $f \in C^1$, $\int |f(x)| dx < \infty$ and $f \in C^2$, $\int |f(x)|^2 dx < \infty$
- Ω is the frequency or fourier domain.
- $F(\Omega)$ contains information about the frequency Ω.
- $F(\Omega) =$ Magnitude spectrum $\times$ Phase spectrum $= |F(\Omega)| e^{j\phi(\Omega)}$
- Inverse fourier transform is

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\Omega) e^{j\Omega x} d\Omega$$

- A few examples are

$$f(x) = \delta(x) \implies F(\Omega) = \int \delta(x) e^{-j\Omega x} dx = 1$$

$$f(x) = \delta(x - x_0) \implies F(\Omega) = e^{-j\Omega x_0}$$

$$f(x) = c \implies F(\Omega) = 2\pi \delta(\Omega)$$

$$f(x) = \cos \Omega_0 x \implies F(\Omega) = \pi[\delta(\Omega - \Omega_0) + \delta(\Omega + \Omega_0)]$$

$$f(x) = A, \; x \in [-\frac{X}{2}, \frac{X}{2}] \implies F(\Omega) = AX sinc(\frac{\Omega X}{2})$$

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} \implies F(\Omega) e^{-\frac{\sigma^2 \Omega^2}{2}}$$

- Properties of Fourier Transform

  1. Consider the function $g(x) = f(x - x_0)$

$$G(\Omega) = \int f(x - x_0)e^{-j\Omega x}dx$$
$$\implies G(\Omega) = e^{-j\Omega x_0}F(\Omega)$$

  2. Convolution for continuous signal

$$y(x) = h(x) * f(X) = \int f(x - x_0)h(x_0)dx_0 = \int h(x - x_0)f(x_0)dx_0$$
$$Y(\Omega) = \int y(x)e^{-j\Omega x}dx = \iint f(x - x_0)h(x_)dx_0e^{-j\Omega x}dx$$
$$= \iint h(x_0)f(x - x_0)e^{-j\Omega x}dxdx_0 = \int h(x_0)F(\Omega)e^{-j\Omega x_0}dx_0 = F(\Omega)H(\Omega)$$

     Convolution in x domain is multiplication in Ω domain, and vice versa.

  3. $f(x) \leftrightarrow F(\Omega) \implies F(x) \leftrightarrow 2\pi f(\Omega)$
  4. $f(x)g(x) \leftrightarrow \frac{1}{2\pi}F(\Omega) * G(\Omega)$
  5. $f(x) \leftrightarrow F(\Omega) \implies \frac{df}{dx} \leftrightarrow j\Omega F(\Omega)$, $\frac{d^n f}{dx^n} \leftrightarrow (j\Omega)^n F(\Omega)$
     Derivative operations are linear shift invariant.

- Fourier transform of the impulse response is referred to as the frequency response of the system(or filter). Impulse response is the fourier transform of dirac delta function.

- **Fourier transform of Impulse Train**
  Impulse train is defined as $\psi_X(x) = \sum_{n=-\infty}^{\infty} \delta(x - nX)$
  This is periodic with period X, $\psi_X(x + X) = \psi_X(x)$
  Periodic functions can be written using a fourier series as

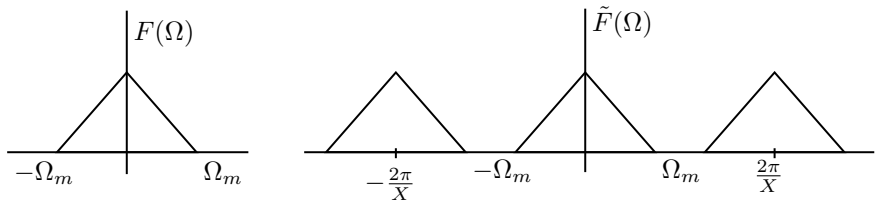
$$\psi_X(x) = \sum_{k=-\infty}^{\infty} c_k e^{j\frac{2\pi}{X}kx}, \ \frac{2\pi}{X} \text{ is the fundamental frequency}$$
$$c_k = \frac{1}{X}\int_{-X/2}^{X/2} \psi_X(x)e^{-j\frac{2\pi}{X}kx}dx = \frac{1}{X}$$
$$\implies \psi_X(x) = \frac{1}{X}\sum_{k=-\infty}^{\infty} e^{j\frac{2\pi}{X}kx}$$
$$\text{Fourier transform } \Psi(\Omega) = \frac{2\pi}{X}\sum_{k=-\infty}^{\infty} \delta(\Omega - \frac{2\pi k}{X})$$

  Fourier transform of an impulse train is also an impulse train.

- **Sampling**

$$\tilde{f}(x) = f(x)\psi_X(x)$$
$$\tilde{F}(\Omega) = \frac{1}{2\pi}F(\Omega) * \Psi(\Omega) = \frac{1}{2\pi}\int F(\Omega_0)\Psi(\Omega - \Omega_0)d\Omega_0 = \frac{1}{X}\sum_{k=-\infty}^{\infty} F(\Omega - \frac{2\pi k}{X})$$

  Two copies will not intersect, i.e., no aliasing if



$$2\Omega_m < \frac{2\pi}{X} \implies \Omega_m < \frac{\pi}{X} \implies X < \frac{\pi}{\Omega_m}$$

- **Continuous band limited** (finite Ω_m)
  Function can be recovered completely from its samples $\rightarrow$ **Sampling Theorem**
  If $f(x)$ is not band limited, can be done by applying low pass filter

$$H(\Omega) = \begin{cases} X, & |\Omega| < \Omega_m \\ 0, & \text{elsewhere} \end{cases}$$

$$\hat{F}(\Omega) = H(\Omega)\tilde{F}(\Omega)$$

  Can take inverse fourier transform of $\hat{F}(\Omega)$ to get back the original signal.

6.3 Discrete Fourier Transform

- The expression of discrete space fourier transform is

$$\tilde{F}(\Omega) = \sum_{n=-\infty}^{\infty} f(n)e^{-jn\omega}$$

  n is discrete however ω is continuous

- Inverse transform of $F(\Omega)$ is periodic with period 2π

$$f(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} F(\omega)e^{jn\omega}d\omega$$

- A few examples are

$$\begin{aligned}
&f(m) = \delta(m) \implies F(\omega) = \sum \delta(m)e^{-jm\omega} = 1 \\
&f(m) = \delta(m - m_0) \implies F(\omega) = e^{-j\omega m_0} \\
&g(m) = f(m - m_0) \implies G(\omega) = F(\omega)e^{-j\omega m_0} \\
&f(m) = 1 \implies F(\omega) = 2\pi \sum_{k=-\infty}^{\infty} \delta(\omega - 2\pi k) \\
&g(m) = f(m) * h(m) \implies G(\omega) = F(\omega)H(\omega)
\end{aligned}$$

Parseval's Theorem

$$\begin{aligned}
\text{Continuous:} \quad & \int |f(t)|^2 dt = \frac{1}{2\pi}\int |F(\Omega)|^2 d\Omega \\
\text{Discrete:} \quad & \sum |f(m)|^2 = \frac{1}{2\pi}\int_{-\pi}^{\pi} |F(\omega)^2| d\omega
\end{aligned}$$

- Discrete Fourier transform has fixed length signals(discrete) and discrete frequency

$$F(k) = \sum_{m=0}^{M-1} f(m)e^{-jm\frac{2\pi}{M}k}, \ k = 0, 1, 2, ..., M-1$$

  also known as M point discrete fourier transform of $f(m)$

- Inverse Discrete Fourier Transform is given as

$$f(m) = \frac{1}{M}\sum_{k=0}^{M-1} F(k)e^{jm\frac{2\pi}{M}k}, \ m = 0, 1, ..., M-1$$

  $F(k + M) = F(k)$ and $f(m + M) = f(m)$, both are periodic with period M

- Implicit in the definition of DFT/IDFT is the fact that we are considering the periodic extension of $f(m)$.

- A few examples are

$$\begin{aligned}
&f(m) = \delta(m) \implies F(\mu) = \sum_{m=0}^{M-1} \delta(m)e^{-j\frac{2\pi}{M}m\mu} = 1 \\
&g(m) = f(m - m_0) \implies G(\mu) = F(\mu)e^{-j\frac{2\pi}{M}\mu m_0} \\
&f(m) = 1 \implies F(\mu) = M\delta(\mu) \\
&f(m) = e^{j\frac{2\pi}{M}\mu_0 m} \implies F(\mu) = M\delta(\mu_0 - \mu), \ \mu_0 = \{0, 1, ..., M-1\} \\
&f(m) = \cos\frac{2\pi}{M}\mu_0 m \implies F(\mu) = \frac{M}{2}[\delta(\mu - \mu_0) + \delta(\mu + \mu_0)]
\end{aligned}$$

- Useful derivation

$$f(m) = \frac{1}{M}\sum_{\mu=0}^{M-1} F(\mu)e^{j\frac{2\pi}{M}m'\mu} \;\; F(\mu) = \sum_{m'=0}^{M-1} e^{-j\frac{2\pi}{M}m'\mu}$$

$$f(m) = \frac{1}{M}\sum_{\mu=0}^{M-1}\sum_{m'=0}^{M-1} f(m')e^{-j\frac{2\pi}{M}m'\mu}e^{j\frac{2\pi}{M}m\mu}$$

Every time we have two summations try to inverse them

$$f(m) = \frac{1}{M}\sum_{m'=0}^{M-1} f(m')\sum_{\mu=0}^{M-1} e^{j\frac{2\pi}{M}\mu(m-m')}$$

$$\sum_{\mu=0}^{M-1} e^{j\frac{2\pi}{M}\mu(m-m')} = \begin{cases} M, \; m = m' \\ 0, \; m \neq m' \end{cases} \quad = M\delta(m-m')$$

$$f(m) = \frac{1}{M}\sum_{m'=0}^{M-1} f(m')M\delta(m-m') = f(m)$$

- **Conjugate Symmetry** for real $f(m)$

$$F(M-\mu) = F^*(\mu) \; 0 \leq \mu \leq M-1$$

$$F(M-\mu) = \sum_{m=0}^{M-1} f(m)e^{-j\frac{2\pi}{M}m(\mu-M)} = \sum_{m=0}^{M-1} f(m)e^{-j\frac{2\pi}{M}\mu} = F^*(\mu)$$

- **Circular Convolution**
  Let DFT be $f(m) \leftrightarrow F(\mu)$ and $g(m) \leftrightarrow G(\mu)$
  Now, we try to find IDFT of $F(\mu)G(\mu)$

$$= \frac{1}{M}\sum_{\mu=0}^{M-1} F(\mu)G(\mu)e^{j\frac{2\pi}{M}m\mu} = \frac{1}{M}\sum_{m_1=0}^{M-1}\sum_{m_2=0}^{M-1} f(m_1)g(m_2)\sum_{\mu=0}^{M-1} e^{j\frac{2\pi}{M}(m-m_1-m_2)\mu}$$

$$= \sum_{m_1=0}^{M-1}\sum_{m_2=0}^{M-1} f(m_1)g(m_2)\delta(m-m_1-m_2) = \sum_{m_1=0}^{M-1} f(m_1)g(m-m_1)$$

$$= f \circledast g$$

- Consider a 5 point signal

m	0	1	2	3	4
$f(m)$	1	2	3	4	5

m	-1	0	1
$g(m)$	1	1	1

$$(f \circledast g)(m) = 8\ 6\ 9\ 12\ 10$$

$$(f * g)(m) = 3\ 6\ 9\ 12\ 9$$

- Summary of Fourier Transforms

	Transform	Inverse Transform
Continuous FT	$F(\Omega) = \int_{-\infty}^{\infty} f(x)e^{-j\Omega x}dx$	$f(x) = \frac{1}{2\pi}\int_{-\infty}^{\infty} F(\Omega)e^{j\Omega x}d\Omega$
Discrete Series FT	$F(\omega) = \sum_{m=-\infty}^{\infty} f(m)e^{-j\omega m}$	$f(m) = \frac{1}{2\pi}\int_{-\pi}^{\pi} F(\omega)e^{j\omega m}d\omega$
Discrete FT	$F(\mu) = \sum_{m=0}^{M-1} f(m)e^{-j\frac{2\pi}{M}\mu m}$	$f(m) = \frac{1}{M}\sum_{\mu=0}^{M-1} F(\mu)e^{j\frac{2\pi}{M}\mu m}$

6.4 2D Fourier Transforms

- Now, we will consider x, y space and Ω, λ space
- Radian frequency: $\Omega_r = \Omega^2 + \lambda^2$
- Angle of Orientation: $\tan^{-1} \frac{\lambda}{\Omega}$
- 2D transform is given as

$$F(\Omega, \lambda) = \iint f(x, y) e^{-j2\pi(\Omega x + \lambda y)} dx dy$$

- Inverse transform is given as

$$f(x, y) = \frac{1}{(2\pi)^2} \iint F(\Omega, \lambda) e^{j2\pi(\Omega x + \lambda y)} d\Omega d\lambda$$

- Similarly we can define Discrete and Discrete Space fourier transforms.
- **Displaying the 2D DFT**

  1. Apply FSCS on $|F(u, v)|$ to observe the spectrum
  2. Optionally can apply $\log 1 + |F(u, v)|$ before FSCS

  Where will high frequency coefficients appear in this image

$$F(M-1, N-1) \equiv F(-1, -1) \; F(u, v) = F(u + M, v + N)$$

  For certain signals, $F(\omega) = F(-\omega)$

$$\begin{aligned} &\implies F(u, v) = F(-u, -v) \\ &\implies F(1, 1) = F(-1, -1) = F(M-1, N-1) \end{aligned}$$

  Central pixels in the 2D-DFT image will correspond to high frequency.

- **Centering the DFT**
  In order to display the DC coordinate at the center of the image.

$$\begin{aligned} \text{Let } \hat{f}(m, n) &= (-1)^{m+n} f(m, n) \\ \hat{F}(u, v) &= \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} (-1)^{m+n} f(m, n) e^{j2\pi(\frac{mu}{M} + \frac{nv}{N})} \text{ Replace } (-1)^{m+n} = e^{j\pi(m+n)} \\ &= F(u - \frac{M}{2}, v - \frac{N}{2}) \\ \hat{F}(\frac{M}{2}, \frac{N}{2}) &= F(0, 0) \end{aligned}$$

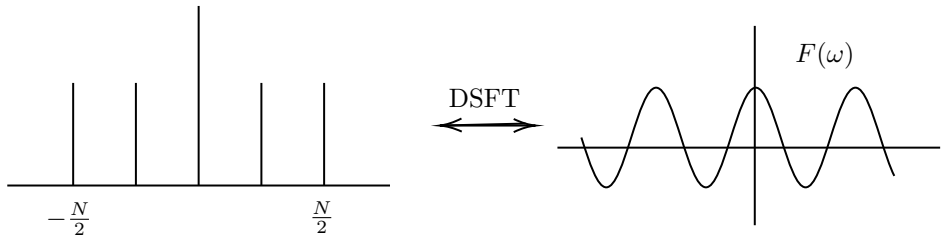
- A few examples

$$\begin{aligned} &f(m, n) = c \implies F(u, v) = c\delta(u, v) \\ &f(m, n) = c\delta(m, n) \implies F(u, v) = c \\ &f(m, n) = \cos[2\pi(\frac{bm}{M} + \frac{cn}{N})] \implies F(u, v) = \frac{1}{2} MN[\delta(u - b, v - c) + \delta(u + b, v + c)] \\ &f(x, y) = \begin{cases} c, & |x| \le \frac{A}{2}, |y| \le \frac{B}{2} \\ 0, & \text{otherwise} \end{cases} \implies F(\Omega, \lambda) = cAB \; sinc(A\Omega) \; sinc(B\lambda) \\ &F(\Omega, \lambda) = \begin{cases} c, & |\Omega| \le a, |\lambda| \le b \\ 0, & \text{otherwise} \end{cases} \implies f(x, y) = cab \; sinc(ax) \; sinc(by) \\ &f(x, y) = ke^{-\frac{1}{2\sigma^2}(x^2 + y^2)} \implies F(\Omega, \lambda) = e^{-2\pi^2\sigma^2(\Omega^2 + \lambda^2)} \end{aligned}$$

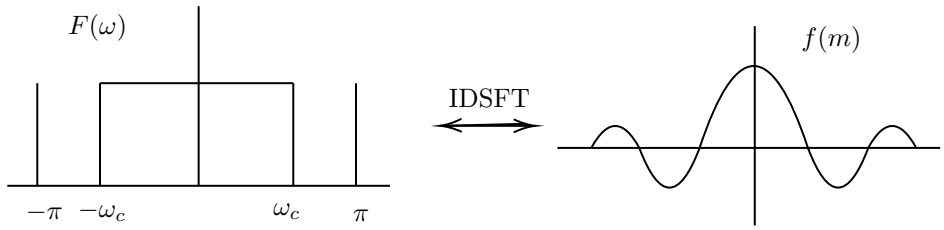
7 Image Filters

7.1 Image Smoothing in frequency domain

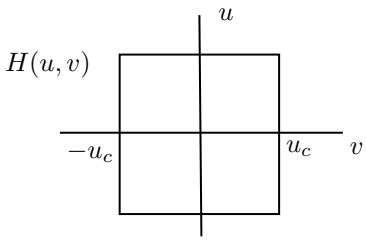
- Fourier transform of $f(m)$



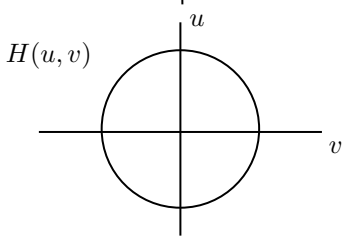
- Ideal low pass filter is given as



- Use of ideal low pass filter causes ringing artifacts, periodic peaks of black intensities.
- Filter centered at white could give higher heights to black due to side lobe peaks.
- We can define 2 different 2D ideal low pass filters



$$H(u,v) = \begin{cases} 1 & \begin{matrix} -u_c < u < u_c \\ -v_c < v < v_c \end{matrix} \\ 0 & \text{otherwise} \end{cases}$$



$$H(u,v) = \begin{cases} 1 & \sqrt{u^2+v^2} \leq r \\ 0 & \text{otherwise} \end{cases}$$

- We can also define a Gaussian low pass filter

$$H(u,v) = \exp\left[-2\pi^2\sigma^2\frac{(u^2+v^2)}{M^2}\right]$$

More height to closer Neighbors. No ringing but cutoff bandwidth degrades gradually instead of instantaneously.

7.2 Image Sharpening in frequency domain

- We can define an ideal high pass filter as

$$H(u,v) = \begin{cases} 1, & \sqrt{u^2+v^2} \geq r \\ 0, & \text{otherwise} \end{cases}$$

- In general, it can be defined as $H_{HPF}(u,v) = 1 - H_{LPF}(u,v)$

- Using the above we can define the Gaussian high pass filter
- Laplacian in continuous domain

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \longleftrightarrow -(\Omega^2 + \lambda^2)F(\Omega, \lambda)$$

Discrete domain: $H(u,v) \longleftrightarrow k(u^2 + v^2)$
Laplacian enhanced image

$$G(u,v) = f(u,v) + cH(u,v)F(u,v)$$

$H(u,v)F(u,v)$ is called the high pass output, can boost high frequencies(edges).
$H(u,v)$ can be replaced with any high pass filter.

7.3 Correlation

- **Cross Correlation**

$$R_{f\omega}(m,n) = \sum_{n'} \sum_{m'} f(m',n')\omega(m+m', n+n')$$

If $\omega = f$, $R_{ff}(m,n) \equiv$ auto correlation function

- **Power Spectral Density (PSD)**
It is the DFT of $R_{f\omega}(m,n)$ or $R_{ff}(m,n)$

$$S_{f\omega}(u,v) = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} R_{f\omega}(m,n) e^{-j2\pi(\frac{um}{M} + \frac{vn}{N})} = W(u,v)F^*(u,v)$$

$$S_{\omega f}(u,v) = F(u,v)W^*(u,v)$$

- We say that ω and f are uncorrelated if $R_{f\omega}(m,n) = 0$

$$\implies S_{f\omega}(u,v) = S_{\omega f}(u,v) = R_{\omega f}(m,n) = 0$$

- We define $S_{\omega\omega}(u,v) = W(u,v)W^*(u,v)$
- If PSD is white $\implies S_{\omega\omega}(u,v) = c \to S_\omega(u,v)$

$$R_{\omega\omega}(m,n) = c\delta(m,n)$$

7.4 Distorted Image Model (Image Restoration)

- We define the model as $g(m,n) = h(m,n) * f(m,n) + \omega(m,n)$, where g is output, h is blur kernel, f is input image and ω is noise.
- We assume ω and f are uncorrelated $R_{f\omega} = 0$
- $h(m,n)$ can be Gaussian blur $h(m,n) = e^{-k(m^2+n^2)}$, or motion blur.
- We will consider three cases

1. $h(m,n) = \delta(m,n)$, in this case output becomes

$$g(m,n) = f(m,n) + \omega(m,n)$$

Noise is white $\implies \delta_\omega(u,v) = c \implies R_{\omega\omega}(m,n) = c\delta(m,n)$
$S_f(u)$: PSD of natural images have power low decay, $\propto u^{-\alpha}$
SNR is defined as $\frac{S_f(u)}{S_\omega(u)}$
Low frequencies have higher SNR and vice versa.
$\implies$ Low pass filters generally work however we loose high frequency components.
We get a denoised image, image looks blurred.
We do not want to completely omit high frequency $\to$ Wiener filter.

2. $\omega(m,n) = 0$, in this case output becomes

$$g(m,n) = h(m,n) * f(m,n)$$

Image Deblurring $G(u,v) = H(u,v)F(u,v)$
Suppose we know $h(m,n)$

$$\implies \hat{F}(u,v) = \frac{1}{H(u,v)} G(u,v) \text{ Inverse filter}$$

Should ideally give $F(u,v)$ but $H(u,v)$ can be zero.

3. $h(m,n)$ is known and $\omega(m,n) \neq 0$

$$g(m,n) = h(m,n) * f(m,n) + \omega(m,n)$$
$$G(u,v) = H(u,v)F(u,v) + W(u,v)$$

Suppose $\hat{F}(u,v) = \frac{1}{H(u,v)}G(u,v) = F(u,v) + \frac{W(u,v)}{H(u,v)}$
If $H(u,v)$ is small $\implies$ Noise gets amplified.

- **Wiener filter (Minimum MSE filter)**
  Find $d(m,n)$ such that for $\hat{f}(m,n) = d(m,n) * g(m,n)$,
  $E = \sum_n \sum_m (f(m,n) - \hat{f}(m,n))^2 = \frac{1}{MN}\sum_u \sum_v |F(u,v) - \hat{F}(u,v)|^2$ is minimized.

$$\begin{aligned} E(u,v) &= |F(u,v) - \hat{F}(u,v)|^2 = |F(u,v) - D(u,v)G(u,v)|^2 \\ &= |F(u,v) - D(u,v)(H(u,v)F(u,v) + W(u,v))|^2 \\ &= |F(u,v)(1 - D(u,v)H(u,v)) - D(u,v)W(u,v)|^2 \\ &= |F(u,v)|^2|1 - D(u,v)H(u,v)|^2 + |D(u,v)|^2|W(u,v)|^2 \end{aligned}$$

  f and ω are uncorrelated

$$\frac{\partial E(u,v)}{\partial D(u,v)} = 0 = |F(u,v)|^2 2|1 - D^*(u,v)H(u,v)||H(u,v)| + 2|D^*(u,v)||W(u,v)|^2$$
$$\implies D^*(u,v) = \frac{|F(u,v)|^2 H(u,v)}{|H(u,v)|^2|H(u,v)|^2 + |W(u,v)|^2} = \frac{S_f(u,v)H(u,v)}{|H(u,v)|^2 S_f(u,v) + S_\omega(u,v)}$$

  1. If $H(u,v) = 0 \implies D(u,v) = 0$ (No instability)
  2. Also no noise amplification

$$D(u,v) = \frac{H(u,v)}{|H(u,v)|^2 + (\frac{S_\omega(u,v)}{S_f(u,v)}) \to \text{SNR}^{-1}}$$

  3. $h(m,n) = \delta(m,n)$

$$D(u,v) = \frac{S_f(u,v)}{S_f(u,v) + S_\omega(u,v)}$$

- **Constrained Least Squares**
  Consider the distorted image $g = h * f + \omega$
  Need to find $\hat{f}$ as $\min \sum\sum(g - h * f)^2$ such that $\sum\sum(\nabla^2 f)^2 \leq c$
  We can rewrite this as $\sum\sum(g - h * f)^2 + \lambda\sum\sum(\nabla^2 f)^2$
  Corresponding function in frequency domain $\sum\sum|G(u,v) - H(u,v)F(u,v)|^2 + \lambda\sum\sum|P(u,v)|^2|F(u,v)|^2$
  where fourier transform of $\nabla^2 f(m,n) = P(u,v)F(u,v)$

$$\implies \hat{F}(u,v) = \frac{H(u,v)G(u,v)}{|H(u,v)|^2 + \lambda|P(u,v)|^2}$$

7.5 Mean and Median Filters

- Consider the neighborhood of pixels $\{y_1, ..., y_N\}$

$$y_i = x + z_i, i = 1, 2, ..., N$$

  We want to denoise x, z_i is stochastic iid

1. Consider $z_i \sim N(0, \sigma_z^2)$
   Goal: Estimate x given $y_1, ..., y_N$
   Maximum Likelihood Estimate, $f(y_1, ..., y_N|x)$ find what x maximizes this

$$\hat{x}_M = \arg\max_x f(y_1, ..., y_N|x) = L$$
$$L = f(y_1, ..., y_N|x) = \prod f(y_i|x) - \prod \frac{1}{\sqrt{2\pi}\sigma_z} e^{-\frac{1}{2}(\frac{y_i - x}{\sigma_z})^2}; f(y_i|x) = N(x, \sigma_z)$$
$$\log L = k - \sum \frac{1}{2}(\frac{y_i - x}{\sigma_z})^2; k = N\log(\frac{1}{\sqrt{2\pi}\sigma_z})$$
$$\hat{x} = \frac{1}{N}\sum y_i (\textbf{Mean filter})$$

2. z_i are iid Laplacian with $f(z) = k\exp[-\frac{|z|}{\sigma}]$ and $E(z) = 0$
   Good for impulse noise $\equiv$ Sudden large noise
   Laplacian distribution can better explain impulse noise better as compared to Gaussian distribution.

$$L = f(y_1, ..., y_N|x) = \prod f(y_i|x) = \prod k\exp -\frac{|y_i - x|}{\sigma}; y_i \sim \text{ Laplacian with } E[y_i] = x$$
$$\log L = N\log k - \sum \frac{|y_i - x|}{\sigma}$$
$$\hat{x} = \arg\min_x \sum |y_i - x| (\textbf{Median filter})$$

This result is obtained from Order Statistics

- **Order Statistics**
  Suppose $y_1, ..., y_N$

$$\begin{aligned} y_{(1)} &= \min\{y_1, ..., y_N\} \\ y_{(2)} &= \text{ second } \min\{y_1, ..., y_N\} \\ &... \\ y_{(N)} &= \max\{y_1, ..., y_N\} \\ &\implies y_{(1)} \leq y_{(2)} \leq ... \leq y_{(N)} \\ \sum|y_i - x| &= \sum|y_{(i)} - x| \end{aligned}$$

  1. Claim: $y_{(1)} \leq x^* \leq y_{(N)}$
  2. Assume that N is odd
     $x^* = y_{(\frac{N+1}{2})} \leftarrow$ Median $\{y_1, ..., y_N\}$
     We can prove by rewriting summation in pairs $(|y_{(1)} - x| + |y_{(N)} - x|) + ... + |y_{(\frac{N+1}{2})} - x|$
     $|y_{(1)} - x| + |y_{(N)} - x| = x - y_{(1)} + y_{(N)} - x = y_{(N)} - y_{(1)}$
     then by 1 we say $y_{(2)} \leq x^* \leq y_{(N-1)}$ and keep going
  3. Assume that N is even
     We follow similar logic as 2 and end up with x^* lies between $y_{(\frac{N}{2})}$ and $y_{(\frac{N}{2}+1)}$. Summation will be same for any x.
     In particular, we take $x^* = \frac{y_{(\frac{N}{2})} + y_{(\frac{N}{2}+1)}}{2}$.

7.6 Bilateral Filter

- We know that an ideal low pass filter causes a jump between low intensity to high intensity at edge resulting in blur.
- Gaussian filter $J_G(i,j) = \sum I(m,n)G(i-m, j-n)$

$$G(i,j) = K_G \exp[\frac{-(i^2 + j^2)}{2\sigma_G^2}]$$

  Weight decreases with spatial distance.

- Bilateral filter also takes into account luminescence distance

$$\begin{aligned} J(i,j) &= \frac{1}{K_{ij}} \sum I(m,n)G(i-m, j-n)H(I(i,j) - I(m,n)) \\ H(k) &= K_H \exp[\frac{-k^2}{2\sigma_H^2}] \\ K_{ij} &= \sum G(i-m, j-n)H(I(i,j) - I(m,n)) \end{aligned}$$

  This is clearly not a linear filter.

- Bilateral filter combines both spatial and luminescence distance.
- It can be extended to color images

$$k^2 \to [(I^R(i,j) - I^R(m,n))^2 + (I^G(i,j) - I^G(m,n))^2 + (I^B(i,j) - I^B(m,n))^2]$$

7.7 Image Resizing

- **Spatial Downsizing**: $D \geq 1$
  D: subsample in each dimension by D

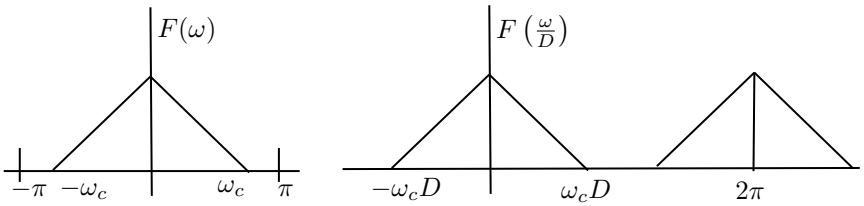
$$h(m,n) = f(mD, nD)$$

Consider the 1D case

$$h(m) = f(mD)$$
$$H(\omega) = \sum h(m)e^{-j\omega m} = \sum f(mD)e^{-j\omega m} = \sum f(m)x(m)e^{-j\omega \frac{m}{D}}$$
$$x(m) = \begin{cases} 1, & m = rD,\ r \in Z \\ 0, & \text{otherwise} \end{cases}$$

DFT of $x(m) \equiv$ periodic D length signal with 1 followed by $D-1$ zeros

$$X(u) = \sum_{m=0}^{D-1} x(m)e^{-j\frac{2\pi}{D}mu} = 1$$
$$\implies x(m) = \frac{1}{D} = \sum_{u=0}^{D-1} X(u)e^{j\frac{2\pi}{D}mu} = \frac{1}{D}\sum_{u=0}^{D-1} e^{j\frac{2\pi}{D}mu}$$
$$H(\omega) = \frac{1}{D}\sum_{m}\sum_{u=0}^{D-1} f(m)e^{j\frac{2\pi}{D}mu}e^{-j\frac{\omega m}{D}}$$
$$\implies H(\omega) = \frac{1}{D}\sum_{u=0}^{D-1} F(\frac{\omega - 2\pi u}{D})$$



No overlapping/aliasing if $2\omega_c D < 2\pi \implies \omega_c < \frac{\pi}{D}$.
Natural images are not strictly band limited.
We can introduce an antialiasing filter, low pass filter with cut-off frequency $\frac{\pi}{D}$.

- **Spatial Upsampling**

$$h(m) = \begin{cases} f(\frac{m}{D}), & m = rD,\ r \in Z \\ 0, & \text{elsewhere} \end{cases}$$

Inserting $D-1$ zeros after each sample.

$$H(\omega) = \sum h(m)e^{-jm\omega} = \sum f(mD)e^{-jmD\omega} = \sum f(m)e^{-jmD\omega} = F(D\omega)$$

Upsampling compresses the spectrum by a factor of D.
High frequency artifacts are introduced due to replicas that appear in $[-\pi, \pi]$.
Can remove high frequency artifacts by applying a low pass filter.

8 Edge Detection

8.1 Introduction

- Edges are **discontinuities in intensity**.
- They can boundaries of objects, boundaries of material properties or boundaries of lighting.
- Edges are sudden, sustained changes in image intensity that extend along a contour.
- Edge is a boundary between two regions with relatively distinct gray level properties.
- Edge detection is used as a precursor to most practical image analysis tasks. Many computer vision algorithms use detected edges as fundamental processing primitives.
- Enormous information is contained in image edges. An image can largely be recognized from its edges alone.
- An edge requires much less storage space than the image itself. It is a binary contour plot.

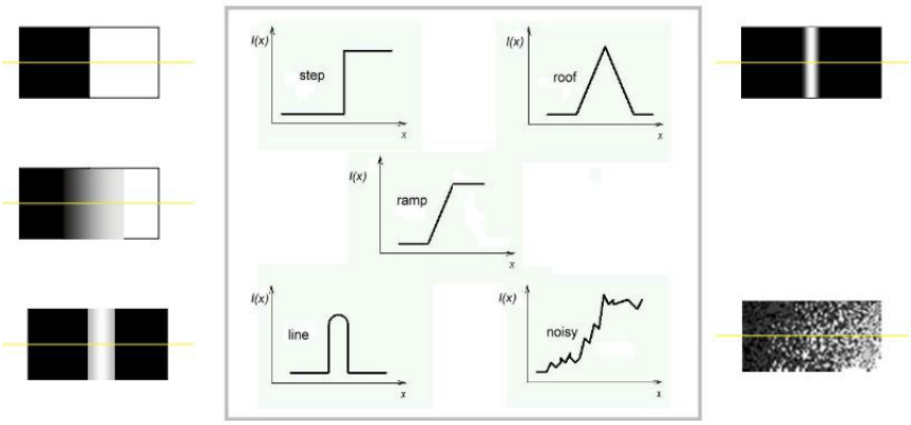


Figure 2: Types of Edges

- Good Edge Detection Algorithms
  1. **Good detection**: Minimize the probability of false positives(spurious edges) and false negatives(missing real edges).
  2. **Good localization**: Detected edges must be as close as possible to the true edges.
  3. **Single response**: Minimize the number of local maxima around the true edge.

8.2 Gradient Based Edge Detection

- Often, points that lie on an edge are detected by detecting the local maxima or minima of the first derivative, and detecting the zero-crossings of the second derivative.
- Compute gradient vector at each pixel by convolving image with horizontal and vertical derivative filters.
  Compute gradient magnitude at each pixel.
  If magnitude at a pixel exceeds a threshold, report a possible edge point.
- Consider the arrangement of pixels about the pixel (i, j)

$$\begin{bmatrix} a_0 & a_1 & a_2 \\ a_7 & [i,j] & a_3 \\ a_6 & a_5 & a_4 \end{bmatrix}$$

  The partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ can be computed as

$$M_x = (a_2 + ca_3 + a_4) - (a_0 + ca_7 + a_6)$$
$$M_y = (a_6 + ca_5 + a_4) - (a_0 + ca_1 + a_2)$$

  The constant c implies the emphasis given to pixels closer to the center of the mask.
- We can define three operators

1. **Prewitt Operator**
$$M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$
M_x and M_y are approximations at (i, j)
2. **Sobel Operator**
$$M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$
M_x and M_y are approximations at (i, j)
3. **Robert Operator**
$$\frac{\partial f}{\partial x} = f(i, j) - f(i+1, j+1)$$
$$\frac{\partial f}{\partial y} = f(i+1, j) - f(i, j+1)$$
This approximation can be implemented by the following masks
$$M_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad M_y = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$
M_x and M_y are approximations at $(i + \frac{1}{2}, j + \frac{1}{2})$

The performance of these three is very similar.

- **Advantages**: Simple, Computationally efficient, works well on "clean" images.
- **Disadvantages**
  1. Extremely noise-sensitive
  2. Requires a threshold: difficult to select - usually requires interactive selection for best results.
  3. Gradient magnitude estimate will often fall above threshold over a few pixels distance from the true edge. So, the detected edge are often a few pixels wide.
  4. This usually requires some kind of "edge thinning" operation - usually heuristic.
  5. The edge contours are often broken - gaps appear.
  6. This requires some kind of "edge linking" operation - usually.
- Noise is a **huge problem** for this type of edge detector.
- Differentiation always emphasizes high frequencies, such as noise.
- Reduce the effects of noise - first smooth with a low pass filter.
- A large filter reduces noise, but worsens localization(i.e., it adds uncertainty to the location of the edge) and vice versa.
- We are essentially doing $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x} h) * f$, from derivate theorem for convolution.
- For further discussion we will consider Gaussian smoothing filter.
- **Effect of Gaussian(sigma value)**
  Larger values: larger scale edges detected
  Smaller values: Finer features detected.
- **Second Derivative Operators**
  First derivative results in detection of too many edge points.
  At edge points, peak in the first derivative, and zero crossing in second derivative.
- **Laplacian Operator**
$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad \nabla^2 \approx \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$
Signals the presence of an edge when the output of the operator makes a transition through zero.

- Need to filter out the noise before edge enhancement.
- **Laplacian of Gaussian**: Combines Gaussian filtering with the Laplacian for edge detection.

$$h(x,y) = \nabla^2[g(x,y) * f(x,y)] = [\nabla^2 g(x,y)] * f(x,y)$$
$$\text{Mexican hat operator: } \nabla^2 g(x,y) = (\frac{x^2+y^2-2\sigma^2}{\sigma^4})e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Following two methods are mathematically equivalent

1. Convolve the image with a Gaussian smoothing filter and compute the Laplacian of the result.
2. Convolve the image with the linear filter that is the Laplacian of the Gaussian filter.

The effect of sigma is similar to Gaussian
Larger values: larger scale edges detected
Smaller values: Finer features detected.

- **Difference of Gaussians**: The Laplacian of Gaussian can be approximated by the difference between two Gaussian functions.

$$\nabla^2 G \approx G(x,y;\sigma_1) - G(x,y;\sigma_2)$$

Ratio $\frac{\sigma_1}{\sigma_2}$ for best approximation is about 1.6, some people like $\sqrt{2}$.

8.3 Canny Edge Detection

- **Step 1**: Smooth the image with a Gaussian filter.

$$S[i,j] = G[i,j;\sigma] * I[i,j]$$

- **Step 2**: Compute the gradient magnitude and orientation using finite difference approximations for the partial derivatives.

$$M[i,j] = \sqrt{P[i,j]^2 + Q[i,j]^2}$$
$$\theta[i,j] = \arctan(Q[i,j], P[i,j])$$

- **Step 3**: Apply nonmaxima suppression to the gradient magnitude.
- To identify edges, broad ridges in M must be thinned so that only the magnitudes at the points of greatest local change remain.
- Nonmaxima suppression thins the ridges in $M[i,j]$ by suppressing all values along the line of the gradient that are not peak values of a ridge.

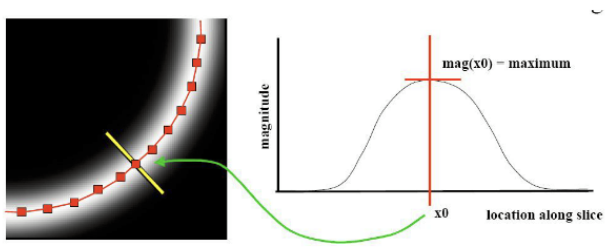


Figure 3: Non-Maximum Suppression

We want to mark points along curve where the magnitude is largest.
We can do this by looking for a maximum along a 1D intensity slice normal to the curve.
These points should form a one-pixel wide curve.

- **Step 4**: Use the double thresholding algorithm to detect and link edges.
Typical procedure to reduce the number of false edge fragments: threshold $N[i,j]$. All values below the threshold are changed to zero.
If threshold too low: some false edges(false positives)
If threshold too high: portions of actual contours may be missing(false negatives).
Selecting the proper threshold is difficult and involves some trial and error.

- **Hysteresis thresholding**: Allows us to apply a high and low threshold.
  Keep both a high threshold H and low threshold L.
  Any edges with strength $< L$ are discarded.
  Any edge with strength $> H$ are kept.
  Any edge P with strength between L and H is kept only if there is a path of edges with strength $> L$ connecting P to an edge of strength $> H$.
  In practice, this thresholding is combined with edge linking to get connected contours.

- **Performance Evaluation**: 3 types of errors, Missing valid edges, Errors in localizing edges, Classification of noise as edges.

$$\text{Pratt's figure of merit } FM = \frac{1}{\max(I_A, I_I)} \sum_{i=1}^{I_A} \frac{1}{1 + \alpha d_i^2}$$

  I_A, I_I, d, and a: detected edges, ideal edges, distance between actual and ideal edges, and a design constant used to penalize displaced edges.
  Usually evaluated on synthetic images by adding random noise in images.

8.4 Hough Transform

- Elegant method for direct object recognition.
- Edges need not be connected, Complete object need not be visible.
- **Key idea**: Edges VOTE for the possible model.
- **Difficulties**: How big should the cells be? (too big, and we merge quite different lines; too small, and noise causes lines to be missed)
- **Finding Circles**: Equation of circle $(x_i - a)^2 + (y_i - b)^2 = r^2$.
  If radius is known then we have 2D Hough space, Accumulator array $A(a, b)$
  If radius is not known then we have 3D Hough Space, Accumulator array $A(a, b, r)$.
- Gradient information can save a lot of computation: Edge location (x_i, y_i), Edge direction ϕ_i

$$a = x - r\cos\phi$$
$$b = y - r\sin\phi$$

- Works on disconnected edges, Relatively insensitive to occlusion, effective for simple shapes
- Handling inaccurate edge locations: Increment Patch in Accumulator rather than a single point.
- Gradient magnitude is isotropic
  $\implies$ take directional derivative in any 2 perpendicular directions, x' and y' and M remains same.
  We can detect edges regardless of orientation.
  Suppose $x', y' \to$ alternate coordinate system rotated by angle ϕ relative to x, y.
  Let $n_{x'}, n_{y'}$ be unit vectors in x', y' directions respectively.

$$\exists x' = \nabla f(x,y)^T n_{x'} = \begin{bmatrix} \exists x(x,y) & \exists y(x,y) \end{bmatrix} \begin{bmatrix} \cos\phi \\ \sin\phi \end{bmatrix} = \exists x \cos\phi + \exists y \sin\phi$$
$$\exists y' = \nabla f(x,y)^T n_{y'} = \begin{bmatrix} \exists x(x,y) & \exists y(x,y) \end{bmatrix} \begin{bmatrix} -\sin\phi \\ \cos\phi \end{bmatrix} = -\exists x \sin\phi + \exists y \cos\phi$$
$$M'^2(x,y) = \exists x'^2 + \exists y'^2 = (\exists x \cos\phi + \exists y \sin\phi)^2 + (-\exists x \sin\phi + \exists y \cos\phi)^2$$
$$= \exists x^2 + \exists y^2 = M^2(x,y) \implies \text{isotropic}$$

  $\exists$ means derivative.

8.5 Harris Corner Detector

- **Template Matching**: Use normalized cross-correlation map to detect matched part.
  Works satisfactorily when matching corresponding regions related mostly by translation.
- **Local Features**: main components
  Detection: Identify the interest points.
  Description: Extract feature descriptor surrounding each point.
  Matching: Determine correspondence between descriptors.

- **Local Features**: desired properties
  Repeatability: Can be found despite geometric and photometric transformations. We have to be able to run the detection procedure *independently* per image.
  Saliency: Distinctive description. Reliably determine which point goes with which. Must provide some invariance to geometric and photometric differences.

- **Patch Matching**: Elements to be matches are image patches of fixed size. Task is to find the best(most similar) patch in a second image.

- **Corners**: Intuitively, junctions of contours. Generally more stable features over changes of viewpoint. Intuitively, large variations in the neighborhood of the point in all directions. They are good features to match.

- "flat" region: no change in all directions.

- "edge": no change along the edge direction.

- "corner": significant change in all directions.

- **Corner Response Map**
  R depends only on eigenvalues of M.
  R is large for a corner.
  R is negative with large magnitude for an edge.
  $|R|$ is small for a flat region.

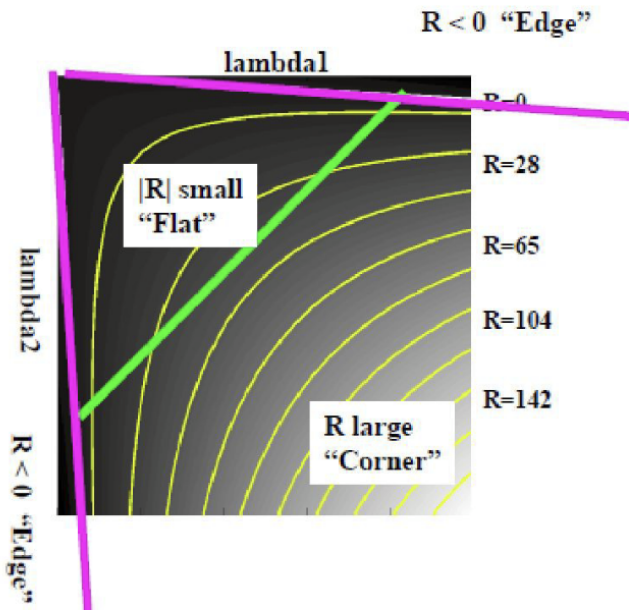


Figure 4: Corner Response Map

- Since, derivatives are invariant to scaling. Harris corner detection is also invariant to intensity change.
  Affine Intensity Change: Partially invariant to affine intensity change.

- Derivatives and window function are shift invariant, corner location is covariant w.r.t translation.

- Second moment ellipse rotates, but its shape remains the same, corner location is covariant w.r.t rotation.

- Corner location is **not** covariant to scaling.

8.6 Blob Detection

- Laplacian of Gaussian
  Edge will be a ripple.
  Blob will be a superposition of two ripples.

- **Spatial selection**: the magnitude of the Laplacian response will achieve a maximum at the center of the blob, provided the scale of the Laplacian is "matched" to the scale of the blob.

- At what scale does the Laplacian achieve a maximum response to a binary circle of radius r?

- To get maximum response, the zeros of the Laplacian have to be aligned with the circle.
- Therefore, the maximum response occurs at $\sigma = \frac{r}{\sqrt{2}}$

8.7 Scale Invariant Feature Transform

- **No. of features**: Generates large number of features that densely cover the image over the full range of scales and locations.
  For small objects: To detect small objects in cluttered backgrounds requires that at least 3 features from each object for reliable identification.
  Typical image(500×500): $\sim$ 2000 stable features.
  Number of features depends on both image content and choices for various parameters.

- **Step-1**: Scale space extrema detection. Identify scale invariant interest points(i.e., key points).
  Scale space of an image is defined as a function

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \; G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

  To detect stable key point locations in scale space, scale space extrema in the difference of Gaussian function convolved with the image used

$$D(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y) = L(x, y, k\sigma) - L(x, y, \sigma)$$

  DoG images are grouped by octaves (i.e., doubling of σ_0), Fixed number of levels per octave.

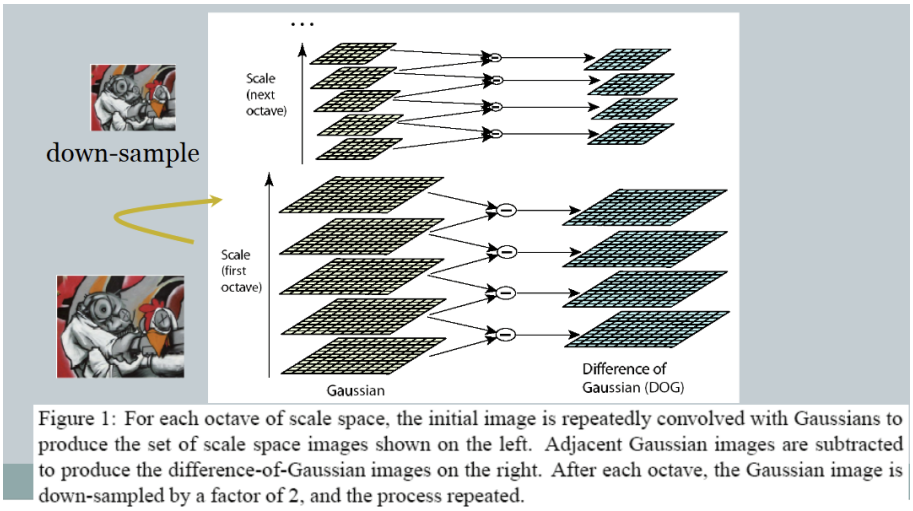


Figure 5: Grouping of Octaves

  Images within each octave are separated by a constant factor k, if each octave is divide in s intervals then $k^s = 2$.

- **Step-2**: Key point localization, Determine accurate location and scale for each interest point. Eliminate "weak" key points.
  Extract local extrema(i.e., minima or maxima) in DoG pyramid.
  Compare each point to its 8neighbors at the same level, 9 neighbors in the level above, and 9 neighbors in the level below(i.e., 26 total).
  Selected only if it is larger or smaller than all of these neighbors.

- **Step-3**: Orientation assignment. Assign one or more orientations to each key point.
  Create histogram of gradient directions, within a region around the key point, at selected scale

$$\begin{aligned} L(x, y, \sigma) &= G(x, y, \sigma) * I(x, y) \\ m(x, y) &= \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2} \\ \theta(x, y) &= a\tan 2 \frac{(L(x+1, y) - L(x-1, y))}{(L(x, y+1) - L(x, y-1))} \end{aligned}$$

  Histogram entries are **weighted** by gradient magnitude and a Gaussian function with σ equal to 1.5 times the scale of the key point.

- **Step-4**: Key point descriptor, Use local image gradients at the selected scale.
  Take a 16×16 window around detected interest point.
  Divide into a 4×4 grid of cells.
  Compute histogram in each cell.

- **Evaluation**
  True Positives(TP) = number of detected matches that are correct.
  False positives(FP) = number of detected matches that are incorrect.

9 Bag of Words Model in Computer Vision

- The steps are given as

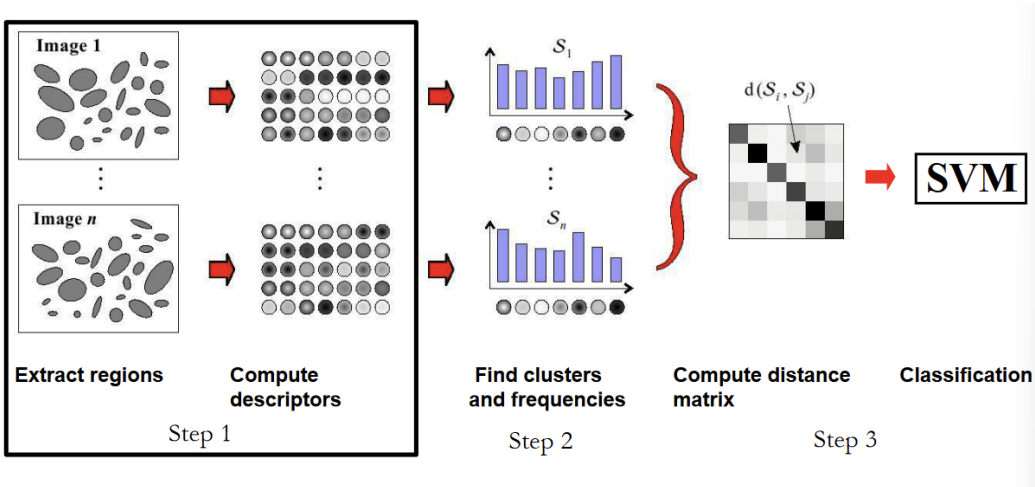


Figure 6: Bag of Words for Computer Vision

- **Documents**: Orderless document representation, frequencies of words form a dictionary.
  Classification to determine document categories.

Figure 7: Document Dictionary

- Obviously in images there are no "words".

- **Texture Recognition**: Texture is characterized by the repetition of basic elements or *textons*.

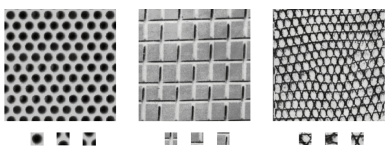


Figure 8: Textons

Now these textons form a dictionary in the format of universal texton dictionary, and then classification can be applied.

- **Visual words**: Extract some local features from a number of images. One example can be the SIFT descriptor space.
- **K means clustering**
  k initial means are randomly generated from the data.
  k clusters created by associating every observation with the nearest mean.
  Centroid of each of the k clusters becomes the new mean.
  Steps 2 and 3 repeated till convergence is reached.
- Idea is that each group of patches will belong to the same visual word.
- We can summarize an image using a bag of visual words.
- This gives excellent results in the presence of background clutter.
- **Pros**: flexible to geometry/deformations/viewpoint, compact summary of image content, provides vector representation, very good results in practice.
- **Cons**: Basic model ignore geometry - must verify afterward or encode via features, background and foreground mixed when bag covers whole image, optimal vocabulary formation remains unclear.

10 CNN - Supervised Classification

- Traditional Approach to Classification

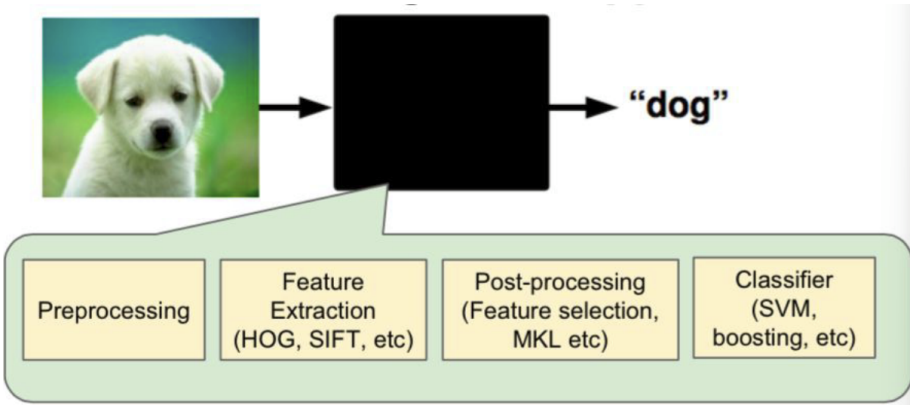


Figure 9: Traditional Classification

- **Limitations**: Most time-consuming in development, what is the best feature?
- The basic idea for CNN is to learn an end-to-end recognition system.
- A non-linear map that takes raw pixels directly to labels.
- Each "layer" can have trainable parameters, their composition makes a highly non-linear system.
- The final layer outputs a probability distribution of categories.
- **ImageNet**: Large Scale Visual Recognition Challenge, 1000 object classes, starting of CNN.
- **Linear Classifier**: Stretch pixels into a column x, then $Wx + b$, this will output scores for different classes.
- Define a **loss function** that quantifies our unhappiness with the scores across the training data.
  A loss function tells how good our current classifier is.
- Given a dataset of examples $\{(x_i, y_i\}_{i=1}^N$, loss over the dataset is a sum of loss over examples

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

- Come up with a way of efficiently finding the parameters that minimize the loss function. (**optimization**).

- **Softmax Classifier**

$$P(Y=k|X=x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$
$$L_i = -\log P(Y=y_i|X=x_i)$$

  we want to maximize log likelihood or want to minimize negative log likelihood of the correct class.

- **Cross Entropy**: Indicates the distance between what the model believes the output distribution should be, and what the original distribution really is.
  Defined as $H(y,p) = -\sum_i y_i \log p_i$
  It is used when node activations can be understood as representing the probability that each hypothesis might be true, i.e., when the output is a probability distribution.
  Thus it is used as a loss function in neural networks which have softmax activations in the output layer.

- **Regularization**: Prevent the model from doing too well on training data

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i,W),y_i) + \lambda R(W)$$
$$\text{L2 regularization: } R(W) = \sum_k \sum_l W_{k,l}^2$$
$$\text{L1 regularization: } R(W) = \sum_k \sum_l |W_{k,l}|$$
$$\text{Elastic net(L1+L2): } R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

  More complex: Dropout, Batch normalization, Stochastic depth, fractional pooling, etc.

- L2 regularization likes to "spread out" the weights.

- Training contains two parts forward propagation and backward propagation.
  Compute loss on small mini-batch
  Compute gradient w.r.t parameters
  Use gradient to update parameters $W \leftarrow W - \eta \frac{dL}{dW}$

- **Follow the slope**: The direction of steepest descent is the negative gradient.

- Taking sum over all data points becomes expensive when N is large, instead we approximate the sum using a **minibatch** of examples, 32/64/128 are common.
  This is also called Stochastic Gradient Descent.

- **Reuse Kernels**: Interesting features(edges) can happen at anywhere in the image. Share the same parameters across different locations, Convolutions with learned kernels.

- **Convolution Layer**: Consider a $32 \times 32 \times 3$ image and $5 \times 5 \times 3$ filter, we convolve over all spatial locations using the filter and end up with a $28 \times 28 \times 1$ activation map.

- If we had used 6 5×5 filters, we will get 6 separate activation maps.
  We stack these up to get a "new image".

- **CovNet**: Sequence of convolution layers, interspersed with activation functions, usually ReLU.

- Essentially speaking, first convolution layer will extract low level features, next one will extract mid level features, and final one will extract high level features.

- First layer filters capture low-level features like oriented edges, blobs.

- **Generic Feature Extractor**: A well-trained CovNet is an excellent feature extractor.
  Chop the network at the desired layer and use the output as a feature representation for a different application.
  Take a pre-trained network and re-train it on a new data: fine-tuning.

- Consider a $N \times N$ image convolving with $F \times F$ filter with *stride*

$$\text{Output size: } \frac{N-F}{stride} + 1$$

- In general, common to see convolutional layers with stride 1, filters of size $F \times F$ and zero-padding with $\frac{F-1}{2}$. (will preserve size spatially)
- Fully Connected layer: Stretch the input image then apply normal neural network functionality.
- **Pooling**: Makes the representations smaller and more manageable, operates over each activation map independently.
  By "pooling" filter responses at different locations we gain robustness to the exact spatial location of features.
  Pooling also subsamples the image, allowing the next layer to look at larger spatial regions.

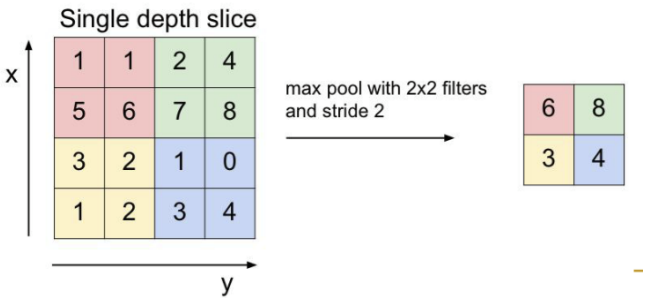


Figure 10: Max Pooling Example

- Non-linear activations

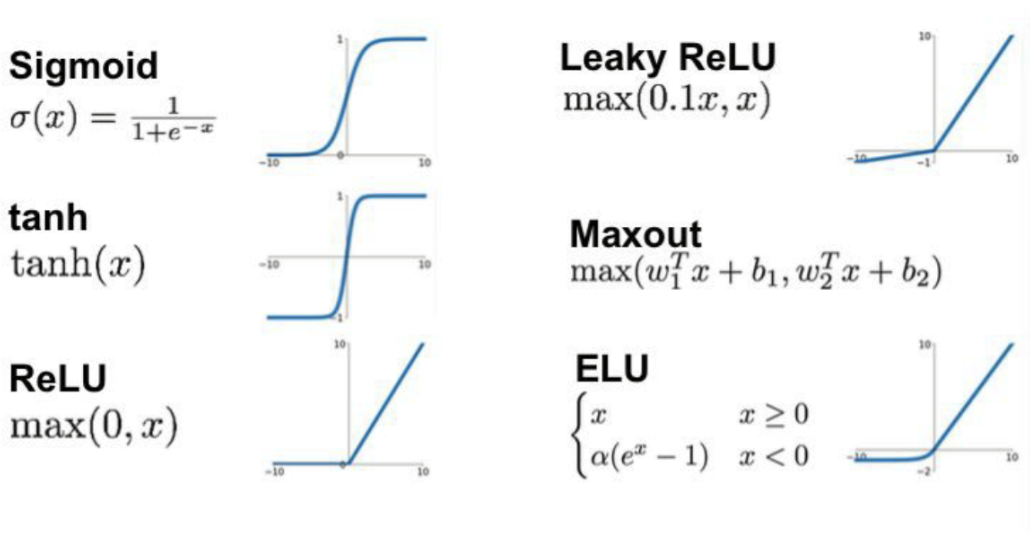
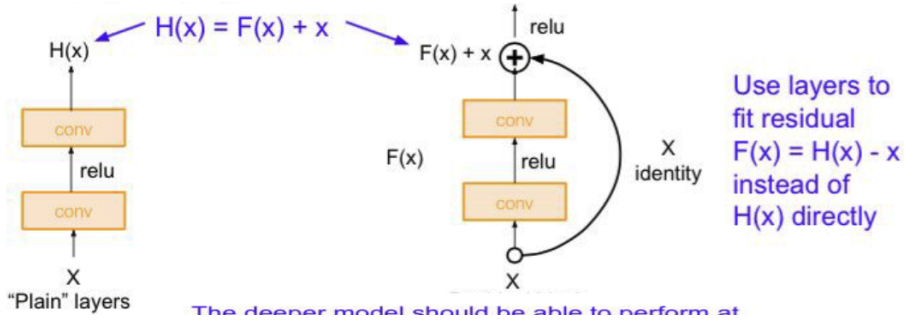


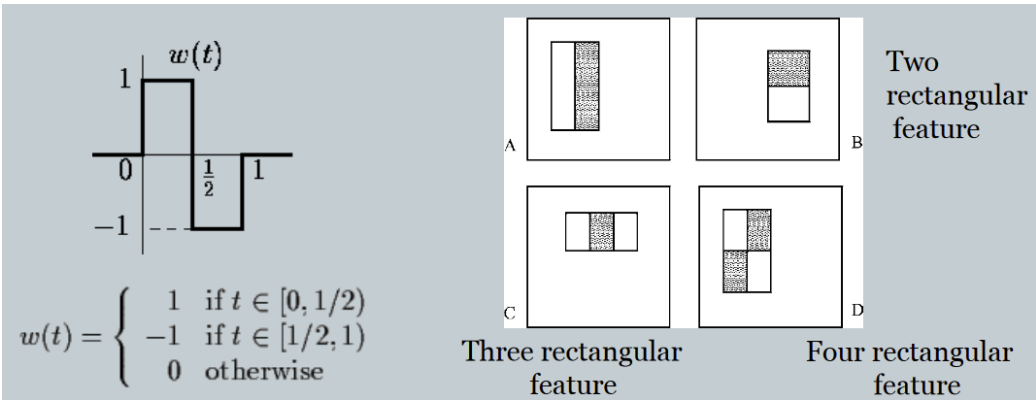
Figure 11: Non-Linear Activation

- **Sigmoid**: Squashes number to range $[0, 1]$. Historically popular since they have nice interpretation as a saturating "firing rate" of a neuron.
  Problems: Saturated neuron "kill" the gradients, Sigmoid outputs are not zero-centered, exp() is a bit compute expensive.
- **Tanh**: Squashes number to range $[-1, 1]$ and is zero centered.
  However, still kills gradients when saturated.
- **ReLU**: Computes $f(x) = \max(0, x)$. Does not saturate, Very computationally efficient, Converges much faster than sigmoid/tanh in practice, actually more biologically plausible than sigmoid.
- **Popular Networks**: AlexNet, VGGNet, ResNet.
- If we continue stacking deeper layers on a "plain" convolutional neural network, the performance becomes worse.
- Instead use network layer to fit a residual mapping instead of directly trying to fit a desired underlying mapping. The deeper model should be able to perform at least as well as the shallower model.



11 Face Detection

- **Biometrics now**: Automatic methods of recognizing/validating the identity of a person based on physiological or behavioral characteristic(s).
- **Why Face?** Natural, non-intrusive, easy-to acquire and use. Ability to acquire signature from non-cooperating subjects.
- **Different FR tasks**: Verification, Identification, Watch List.
- **Face Detection**: First step in any face recognition system. Most FR algorithms assume the face to be detected and accurately cropped. Accurate localization is required for good recognition.
- **Problem**: Given an arbitrary image, the goal is to determine if there are any faces in the image and return the image location and extent of each face.
  Related problems: Facial feature detection, Face tracking, Face localization.
- **Challenges**
  Non-rigid object: Pose, expression variations
  Illumination variations, Variation in scale, location, orientations
  Presence/absence of glasses, beard, mustaches.
  Occlusion in group images.
- **Viola Jones Face Detector**: Two class problem, face/non-face.
  Feature extraction, feature selection, cascade of classifiers.
- **Rectangular features**: Haar like features, simple and easy to compute, efficient detector.
  Feature: White areas subtracted from black ones



- **Integral Image**: Features computed very rapidly using integral image.

$$ii(x,y) = \sum_{x' \le x, y' \le y} i(x',y')$$

$$\text{Cumulative sum: } s(x,y) = s(x,y-1) + i(x,y)$$
$$ii(x,y) = ii(x-1,y) + s(x,y)$$

- Integral image can be computed in one pass, value of any rectangular sum can be computed efficiently.

$$ii(\text{bottom-right}) + ii(\text{top-left}) - ii(\text{top-right}) - ii(\text{bottom-left})$$

- Adaboost used for selecting features and training classifier.
  Boosting: classification scheme that works by combining weak learners into a more accurate classifier.
  Step 1: Given a set of weak classifiers, pick the best one.
  Step 2: Reweight the data so that the inputs where the first classifier has made errors get larger weights.
  Step 3: Now choose the second weak classifier based on the weighted data.
  Step 4: Reweight the data according to the errors and choose the next classifier and so on
  Final Step: Final classifier is a weighted combination of all the weak classifiers.

- **Boosting for Face Detection**: Greedy feature selection process, weak learners based on rectangular features.
  In each round of boosting: evaluate each feature on the examples, select best threshold so that minimum number of examples are misclassifies, choose classifier and reweight examples.
  More features improve classification accuracy.

- **Attentional Cascade: Cascade of Classifiers**
  Improve detection, reduce computation time.
  In an image, majority does not contain face.
  Simple classifiers - reject the majority of subwindows, before complex classifiers are used.
  Most non-face regions are rejected at early stage.

- **Evaluation Metrics**
  Detection rate: Ratio of number of faces correctly detected to number faces determined by a human.
  An image region identified as a face by a classifier is considered to be correctly detected if it covers more than a certain percentage of a face in the image.
  False negatives: faces are missed causing low detection rates
  False positives: image region declared to be face, but it is not.

- **Knowledge Based Method**
  Rule-based methods encode human knowledge of what constitutes a typical face.
  Face has two eyes symmetric to each other, a nose, mouth.
  Usually, the rules capture the relationships between facial features - relative distances and positions.
  Facial features extracted and face candidates identified based on the rule.
  Advantages: Easy to come up with simple rules, works well for frontal faces in uncluttered scenes.
  Disadvantages: Too strict or too general, difficult for non-frontal poses.

- **Template Matching**: Store a template based on edges/regions or on facial contours.
  Templates are usually hand-coded, use correlation to locate faces.
  Advantage: Simple to implement
  Drawback: Difficult to enumerate templates for different poses, cannot effectively deal with variation in scale, pose, and shape.

- **Face detection in video**: An easier problem than still images, use all available cues(motion).
  Advantages: There are multiple images, can integrate information temporally across the video sequence, dynamic information can help.
  Typically, a two-stage system: tracking module and recognition module.

12 Image quality Assessment

- Consider I as reference image and J as distorted image

$$\text{Mean Squared Error: } MSE(I,J) = \frac{1}{MN}\sum_{i=0}^{M-1}\sum_{j=0}^{N-1}(I(i,j)-J(i,j))^2$$

$$\text{Peak Signal to Noise ratio: } PSNR(I,J) = 10\log_1 0\frac{L^2}{MSE(I,J)},\ L \approx 255$$

  PSNR and MSE are monotonically related.

- Structural Similarity Index

$$SSIM(i,j) = L(i,j)C(i,j)S(i,j) = \text{Luminescence Contrast Structure}$$

$$L(i,j) = \frac{2\mu_I(i,j)\mu_J(i,j)+c_1}{\mu_I^2(i,j)+\mu_J^2(i,j)+c_1}$$

$$\mu_I(i,j) = \sum_{q=-P}^{P}\sum_{p=-P}^{P} w(p,q)I(i+p,j+q)$$

$w(p,q)$ is an isotropic unit area, Gaussian weighting function

$$\sum_{q=-P}^{P}\sum_{p=-P}^{P} w(p,q) = 1$$

$\mu_J(i,j)$ is computed similarity

- Contrast Similarity

$$C(i,j) = \frac{2\sigma_I(i,j)\sigma_J(i,j) + c_2}{\sigma_I^2(i,j) + \sigma_J^2(i,j) + c_2}$$
$$\sigma_I^2(i,j) = \sum_{q=-P}^{P}\sum_{p=-P}^{P} w(p,q)[I(p+i,q+j) - \mu_I(i,j)]^2$$

$w(p,q)$ is the same weighting function as before., $\sigma_I^2(i,j)$ is also defined similarity.

- Structure Similarity

$$S(i,j) = \frac{\sigma_{IJ}(i,j) + c_3}{\sigma_I(i,j)\sigma_J(i,j) + c_3}$$
$$\sigma_{IJ}(i,j) = \sum_{p=-P}^{P}\sum_{q=-P}^{P} w(p,q)[I(i+p,j+q) - \mu_I(i,j)] \times [J(i+p,j+q) - \mu_J(i,j)]$$
$$SSIM_{IJ} = \frac{1}{MN}\sum_{j=0}^{N-1}\sum_{i=0}^{M-1} SSIM(i,j) \rightarrow \text{ SSIM map or quality map}$$

13 Karhunen Loeve Transform

- Very similar to PCA, goal is $y = Tx$ where $x \sim \mathcal{N}(0,k)$ and $y \sim \mathcal{N}(0,k')$
  Find T such that k' is diagonal
- k' is a covariance matrix.
- Since k is a covariance matrix, it has an eigen value decomposition

$$k = U\Lambda U^T$$

- **Claim**: $T = U^T$
- This will make $y = U^T x = \Lambda$
- A number of diagonal entries in Λ will be close to zero for image.
- How do we compute KLT for images?
  Take blocks of pixels/patches. Each block is then flattened and treated as a data point.